

Trocando o Controlador Clássico (PID) de um Avião por uma Rede Neural (LSTM)

Imitação de um controlador PI por rede recorrente, com refinamento por DAgger e análise de robustez

Kauê Martins de Souza

Trabalho Final — [disciplina] — [instituição]

Orientação: Prof.^a [Neusa — sobrenome]

Junho de 2026

Material completo (código MATLAB/Simulink, dados e este relatório):

https://github.com/gabimont/pitch_lstm

Resumo

Todo avião precisa de um “piloto automático” que mantenha o nariz na inclinação certa. Tradicionalmente isso é feito por um **controlador clássico** (uma fórmula matemática bem conhecida, o PID). Neste trabalho, trocamos essa fórmula por uma **rede neural** — um programa de computador que *aprende sozinho*, a partir de exemplos. Ensinamos a rede a *imitar* o controlador clássico e depois a colocamos para comandar o avião sozinha, comparando os dois. O resultado tem três partes: (1) quando treinada corretamente, a rede neural funciona tão bem quanto o controlador clássico; (2) a imitação simples, porém, não basta — a rede precisou de um treino extra (chamado *DAgger*) para parar de “balançar” o avião; e (3) mapeamos onde a rede funciona e onde ela falha. A lição final, em uma frase: **a rede neural só é confiável nas situações parecidas com as que ela viu durante o treino**. Fora delas, ela piora — mas, neste estudo, nunca a ponto de descontrolar totalmente o avião.

Sumário

1	O que estamos tentando fazer	2
2	O sistema e a malha de controle	2
2.1	A planta: o modelo da aeronave	2
2.2	Resposta em malha aberta	2
2.3	O controlador PI de referência	3
2.4	A implementação no Simulink	4
3	O que é uma rede neural LSTM?	5
3.1	De um neurônio a uma rede	5
3.2	O ingrediente que falta numa rede comum: memória	5
3.3	As “comportas” que dão nome à LSTM	6
3.4	A arquitetura usada neste trabalho	6
3.5	Como a rede é treinada	6
4	Como a rede neural aprende	8
4.1	Aprender por imitação	8
4.2	Por que só imitar não basta: o DAgger	8

5	Os dados de treinamento	10
5.1	A ideia: o PID gera os dados	10
5.2	Variedade: cobrindo muitas situações	11
5.3	O reforço do DAgger	12
6	Resultado: a rede empata com o controlador clássico	12
7	Onde a rede falha: testando fora do que ela aprendeu	14
8	O que pode ser melhorado	17
9	Conclusão	18

1 O que estamos tentando fazer

O problema é o controle de arfagem (*pitch*) de uma aeronave: manter o ângulo θ numa referência θ_{ref} , atuando no profundor. O laço mede θ , compara com θ_{ref} e calcula o comando de deflexão — tarefa classicamente resolvida por um controlador **PID/PI**, com garantias de estabilidade e desempenho bem estabelecidas. A pergunta deste trabalho é:

*E se, no lugar dessa fórmula clássica, usássemos uma **rede neural** — um programa que aprende sozinho a partir de exemplos?*

Redes neurais são flexíveis e estão por trás de muita tecnologia moderna (reconhecer fotos, traduzir textos). A ideia aqui é ensinar uma rede a controlar o avião e medir, com cuidado, **onde ela funciona, onde falha e por quê**. Tudo foi feito em simulação, no programa MATLAB/Simulink.

2 O sistema e a malha de controle

2.1 A planta: o modelo da aeronave

A planta é a cascata *servo do profundor + dinâmica de arfagem de curto período*, modelada por duas funções de transferência:

$$G_{\text{elev}}(s) = \frac{\delta_e}{\delta_c} = \frac{-0,1}{0,1s + 1}, \quad G_{\text{din}}(s) = \frac{\theta}{\delta_e} = \frac{-3}{s^2 + 2s + 5}. \quad (1)$$

G_{elev} é o servo do profundor (primeira ordem, $\tau = 0,1$ s, mapeando o comando δ_c na deflexão δ_e); G_{din} é a dinâmica de arfagem (segunda ordem, $\omega_n = \sqrt{5} \approx 2,24$ rad/s, $\zeta = 1/\sqrt{5} \approx 0,45$, sub-amortecida). Em série formam a planta vista pelo controlador (de δ_c a θ):

$$G_p(s) = G_{\text{elev}}(s) G_{\text{din}}(s) = \frac{0,3}{(0,1s + 1)(s^2 + 2s + 5)}, \quad (2)$$

sistema de terceira ordem, estável em malha aberta (polos em $s = -10$ e $s = -1 \pm 2j$), com ganho DC $+0,06$. Os dois sinais negativos de (1) se cancelam, de modo que G_p tem ganho positivo — e o controlador, ganhos positivos; o par complexo $-1 \pm 2j$ ($\zeta \approx 0,45$) responde pelo caráter oscilatório da resposta. A realimentação é unitária ($k = 1$), com erro $e = \theta_{\text{ref}} - \theta$ (Fig. 1).

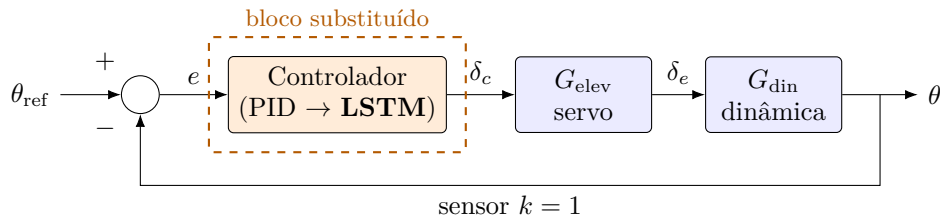


Figura 1: Malha de controle de arfagem, com realimentação unitária. O controlador calcula o comando δ_c a partir do erro $e = \theta_{\text{ref}} - \theta$; o servo G_{elev} produz a deflexão δ_e do profundor e a dinâmica G_{din} a converte no ângulo de arfagem θ . Trocamos apenas o bloco do controlador: o PID clássico passa a ser uma rede neural LSTM.

2.2 Resposta em malha aberta

A resposta ao degrau unitário em δ_c (Fig. 2) exibe o sobressinal do par sub-amortecido e converge para o ganho DC (0,06), não para a entrada: em malha aberta a saída é apenas proporcional ao comando ($\theta \rightarrow 0,06 \delta_c$), sem rastreamento. É a ação integral em malha fechada que estabelece o seguimento de referência (Fig. 3).

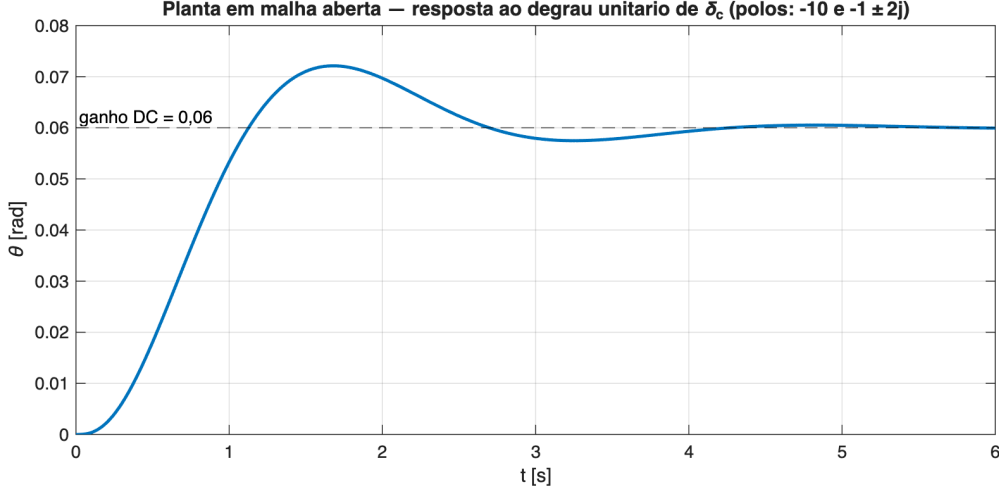


Figura 2: Planta em **malha aberta**: resposta de θ a um degrau *unitário* no comando do elevador ($\delta_c = 1$). A saída oscila (polos $-1 \pm 2j$, $\zeta \approx 0,45$) e se acomoda em 0,06 (o ganho DC), proporcional à entrada. Em malha aberta não há referência nem realimentação: a planta sozinha não leva θ a um valor desejado.

2.3 O controlador PI de referência

O controlador clássico (o “professor”) é um **PI** (Proporcional + Integral). A cada instante ele calcula o comando a partir do erro $e = \theta_{\text{ref}} - \theta$:

$$\delta_c(t) = \underbrace{K_p e(t)}_{\text{proporcional}} + \underbrace{K_i \int_0^t e(\tau) d\tau}_{\text{integral}}. \quad (3)$$

A ação integral confere à malha tipo 1, garantindo erro de regime nulo a referências em degrau. Vale fixar esta parcela: é a integral — dependente de *todo* o histórico de e — que a rede precisará reproduzir a partir apenas de $e(t)$, e que se mostrará o ponto frágil fora do envelope de treino.

Optou-se por PI (sem o termo derivativo K_d) porque (i) a planta já é adequadamente amortecida ($\zeta \approx 0,45$), dispensando o amortecimento da derivada, e (ii) K_d introduziria ganho de alta frequência, sensível a ruído e difícil de imitar a partir somente de $e(t)$.

Sintonia via `pidtune` sobre G_p : $K_p = 11,79$, $K_i = 15,15$. A malha fechada resultante apresenta sobressinal $\approx 1,3\%$, $t_s(2\%) \approx 5,6$ s, $t_r \approx 0,82$ s e erro de regime nulo. Implementação discreta a $T_s = 0,01$ s (100 Hz), com a planta contínua integrada por `ode4` (passo fixo) e o PI discretizado por *Forward Euler* (bloco PID do Simulink).

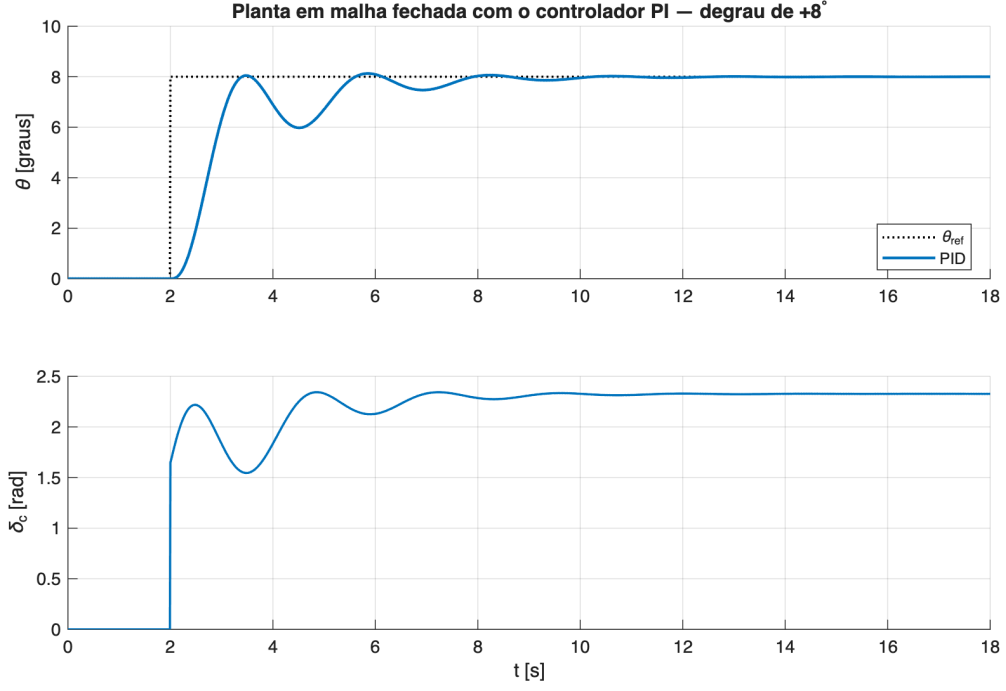


Figura 3: A planta em **malha fechada** com o PI, para um degrau de $+8^\circ$. Ao contrário da malha aberta (Fig. 2), agora a saída θ *rastreia* a referência: sobressinal pequeno ($\approx 1,3\%$), acomodação em $\approx 5,6$ s e **erro de regime nulo** — graças à ação integral. **Abaixo:** o comando δ_c que o controlador envia à planta.

2.4 A implementação no Simulink

A malha foi montada no Simulink (Fig. 4) — é a tradução direta do diagrama da Fig. 1. O bloco `theta_ref_src` gera a referência θ_{ref} ; `sum_e` forma o erro $e = \theta_{\text{ref}} - \theta$; o bloco **Controlador** (aqui o PI discreto, $\text{PI}(z)$) produz o comando δ_c ; e a planta é fechada por G_{elev} (função de transferência Y/U) seguida de G_{din} (implementada em **espaço de estados** $\dot{x} = Ax + Bu$, $y = Cx + Du$, justamente para expor a condição inicial $\theta(0)$ usada nos testes de robustez). A saída θ realimenta o somador, e os blocos `out.*` apenas registram os sinais (θ_{ref} , e , δ_c , θ) para análise posterior.

O ponto central do trabalho aparece aqui de forma literal: os modelos `pitch_PID.slx` e `pitch_LSTM.slx` são *idênticos*, exceto pelo bloco **Controlador** — é exatamente o “bloco substituído” da Fig. 1. Nesta seção ele é o $\text{PI}(z)$; ao trocá-lo pela rede obtém-se o modelo da Fig. 6 (Seção 3.5), cujo interior está detalhado na Fig. 7.

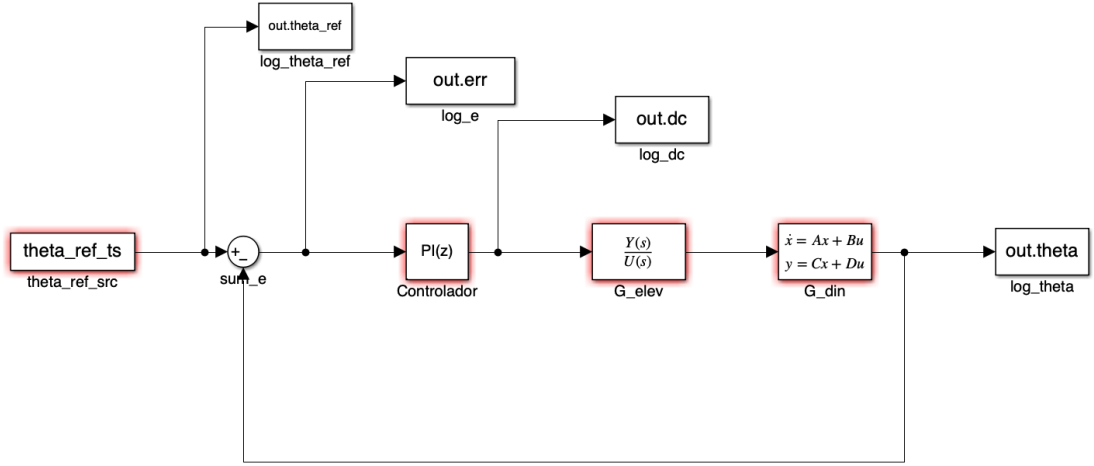


Figura 4: A malha de controle no **Simulink** (modelo `pitch_PID.slx`), realização concreta do diagrama da Fig. 1. Da esquerda para a direita: a referência θ_{ref} , o somador que forma o erro e , o **Controlador** (aqui o bloco $\text{PI}(z)$), o servo G_{elev} e a dinâmica G_{din} em espaço de estados, com a saída θ realimentando a entrada. Os blocos à parte (`out.theta_ref`, `out.err`, `out.dc`, `out.theta`) só registram os sinais para análise.

3 O que é uma rede neural LSTM?

Esta seção apresenta a rede neural utilizada. A exposição apoia-se em analogias com sistemas dinâmicos — a principal delas sendo que **uma LSTM é, em essência, um sistema dinâmico discreto com estado, cujas equações são aprendidas a partir de dados** em vez de deduzidas da física.

3.1 De um neurônio a uma rede

A unidade básica é o **neurônio artificial**: ele recebe alguns números de entrada, multiplica cada um por um **peso**, soma tudo (com um termo constante, o *viés*) e passa o resultado por uma função simples e não linear (uma “curva em S”, por exemplo). Uma **rede neural** é apenas muitos desses neurônios organizados em **camadas**, onde a saída de uma camada alimenta a próxima. Toda a “inteligência” está nos *pesos* — e **treinar** a rede significa, simplesmente, ajustar esses pesos até que a saída dela fique o mais próxima possível da saída desejada (aqui, o comando que o PID daria). É um problema de *regressão*, como ajustar uma curva a pontos, só que com muitos parâmetros.

3.2 O ingrediente que falta numa rede comum: memória

Uma rede neural comum (dita *feedforward*) comporta-se como um **ganho estático não linear**: para a mesma entrada, devolve sempre a mesma saída, sem nenhuma noção de passado. Isso é fatal para o nosso problema, porque o comando do PID **não** depende só do erro atual — o termo integral $K_i \int e$ depende de *todo* o histórico do erro. Uma rede sem memória não teria como reproduzir essa parcela.

A **LSTM** (*Long Short-Term Memory*, “memória de curto-longo prazo”) resolve isso carregando um **estado interno** que ela atualiza a cada instante. Em linguagem de controle, isso é *exatamente* um sistema dinâmico discreto:

$$h[k] = f(h[k-1], e[k]), \quad \delta_c[k] = g(h[k]), \quad (4)$$

onde o vetor h é o **estado** (a “memória”) da rede — o análogo direto do vetor de estados x de uma representação em espaço de estados, ou do acumulador do integrador do PID. A diferença

para um modelo clássico é que as funções f e g não são escritas à mão a partir da física: elas são **aprendidas dos dados**. É essa memória, governada por (4), que permite à rede representar a ação integral acumulada ao longo de um episódio.

3.3 As “comportas” que dão nome à LSTM

O que distingue a LSTM de outras redes com memória é *como* ela decide o que guardar e o que descartar. Cada unidade possui três **comportas** (*gates*) — pequenas válvulas com abertura contínua entre 0 (fechada) e 1 (aberta), cujos ajustes também são aprendidos:

- a **comporta de esquecimento** decide quanto do estado anterior manter;
- a **comporta de entrada** decide quanto da informação nova incorporar;
- a **comporta de saída** decide quanto do estado interno expor na saída.

Na prática, essas comportas permitem que a rede preserve uma informação relevante por centenas ou milhares de passos sem que ela “vaze” — comportamento ideal para representar um acumulador, como a integral do erro. É daí que vem o nome: memória que pode ser tanto de *curto* quanto de *longo* prazo, conforme a tarefa exigir.

3.4 A arquitetura usada neste trabalho

A rede recebe *um* número por instante (o erro $e[k]$) e devolve *um* número (o comando $\delta_c[k]$). Entre a entrada e a saída há duas camadas LSTM empilhadas, com 64 e 32 unidades, seguidas de uma camada *totalmente conectada* que combina linearmente o resultado num único valor (Fig. 5).

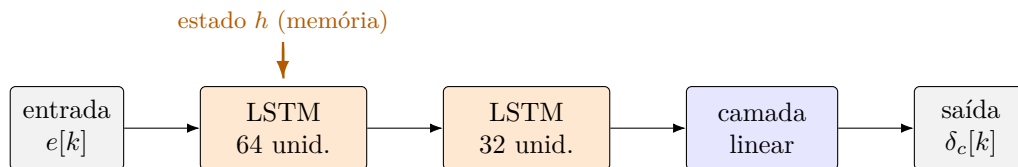


Figura 5: Arquitetura da rede: um erro entra por instante, passa por duas camadas LSTM (que carregam o estado/memória h) e por uma camada linear, e sai um comando. É a versão “aprendida” do controlador — compare com a Eq. (4).

Por que *duas* camadas e justamente esses tamanhos? Em testes preliminares, camadas menores (32 e 16 unidades) não conseguiam manter o acumulador integral com precisão por milhares de passos: a resposta à *rampa* (dominada pela parcela $K_i \int e$) ficava pobre. Com $64 \rightarrow 32$ a rede ganha “espaço” suficiente para representar bem a ação integral, sem ficar grande a ponto de decorar os dados.

3.5 Como a rede é treinada

Treinar é resolver um problema de otimização: ajustar os pesos para minimizar o **erro quadrático médio** (MSE) entre o comando que a rede produz e o comando que o PID produziu, somado sobre todos os exemplos. Os principais elementos do treino:

- **Normalização.** Antes de entrar na rede, o erro é reescalado para ter média 0 e desvio 1 (*z-score*); a saída é “desnormalizada” de volta para graus de comando. Isso só deixa os números numa escala confortável para o treino — na malha de controle, aparece como pequenos blocos de ganho e viés em torno da LSTM.

- **Otimizador.** Usamos o algoritmo *Adam*, com até 350 *épocas* (passagens pelos dados) e taxa de aprendizado inicial 3×10^{-3} , reduzida pela metade periodicamente para refinar o ajuste no fim.
- **Validação e parada antecipada.** Separamos os episódios em 70 % para treino, 15 % para validação e 15 % para teste. Guardamos a versão da rede com *melhor* desempenho na validação (*early stopping*): isso evita a “decoreba” (*overfitting*), em que a rede memoriza os exemplos mas vai mal em casos novos.
- **Episódio inteiro.** Treinamos com a sequência completa de cada manobra (não com janelas curtas), para que o estado da LSTM acumule a integral desde $t = 0$ — exatamente como ela opera depois na malha fechada.

Na malha de controle, a rede treinada substitui o bloco $PI(z)$ por um subsistema **Controlador**: o modelo `pitch_LSTM.slx` (Fig. 6) é *idêntico* ao `pitch_PID.slx` (Fig. 4), trocando apenas esse bloco. Por dentro (Fig. 7), o subsistema é a arquitetura da Fig. 5 embrulhada pela normalização descrita acima: o erro e entra, é normalizado, atravessa a rede recorrente operando em modo *Stateful Predict* (que mantém o estado/memória h entre os passos da simulação) e é desnormalizado de volta para o comando δ_c em graus.

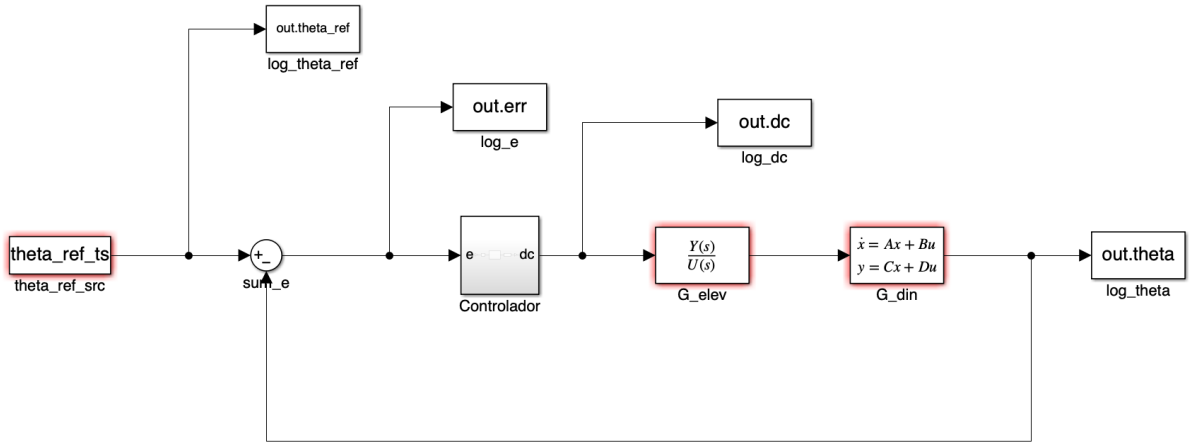


Figura 6: A mesma malha da Fig. 4, agora no modelo `pitch_LSTM.slx`: tudo é igual ao caso do PID, exceto o bloco **Controlador**, que deixou de ser o $PI(z)$ e passou a ser o subsistema da rede neural (detalhado na Fig. 7). É a materialização do “bloco substituído” da Fig. 1.

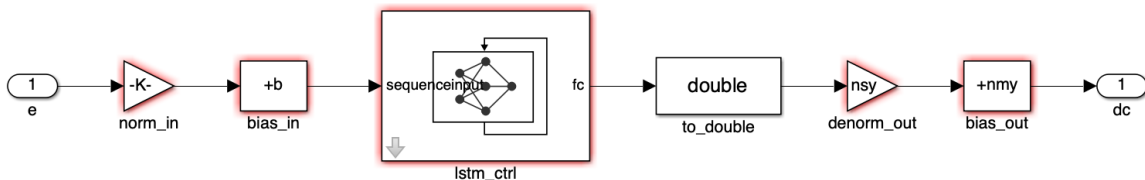


Figura 7: O subsistema **Controlador** do `pitch_LSTM.slx` por dentro — o “bloco substituído” da Fig. 1. O erro e é normalizado (`norm_in`: ganho $-K$ e viés $+b$, o z -score), passa pela rede recorrente (`lstm_ctrl`: `sequenceinput`→LSTM→`fc`) em *Stateful Predict*, é convertido para `double` e desnormalizado (`denorm_out`/`bias_out`) de volta para o comando δ_c . É a Eq. (4)/Fig. 5 com a (des)normalização ao redor.

A próxima seção descreve o procedimento de treinamento da rede para controlar o avião.

4 Como a rede neural aprende

4.1 Aprender por imitação

A estratégia é simples de explicar: deixamos o controlador clássico comandar o avião em milhares de situações simuladas e **anotamos tudo** — em cada instante, qual era o erro e qual foi o comando dado. Isso vira uma grande tabela de exemplos “situação $\rightarrow$ comando”. A rede neural então treina para **reproduzir** esses exemplos: dado o histórico do erro, ela tenta dar o mesmo comando que o professor daria. É o que se chama *aprender por imitação*.

Usamos uma rede do tipo **LSTM** [3], descrita na Seção 3: a característica que importa aqui é que ela tem **memória** (o estado interno h) — lembra do que aconteceu nos instantes anteriores. Isso é essencial porque o professor também tem memória (aquela parcela do “acúmulo do erro”, a integral). Sem memória, a rede não conseguiria imitá-lo.

4.2 Por que só imitar não basta: o DAgger

Aqui está o ponto mais importante do trabalho, e vale uma analogia:

Imagine ensinar alguém a dirigir mostrando *só* vídeos de um motorista experiente numa estrada perfeita. A pessoa decora aquilo. Mas, quando assume o volante e comete um pequeno desvio, ela se vê numa situação que **nunca apareceu nos vídeos** — e não sabe como reagir. O pequeno erro vira um erro maior, e ela perde o controle.

Foi exatamente o que aconteceu com a rede de imitação pura. No papel ela ia muito bem (copiava o comando do PID com fidelidade alta). Mas, ao assumir o comando do avião, qualquer errinho a levava a uma situação que o PID nunca havia criado — e ali ela não sabia o que fazer. O resultado é a curva vermelha da Figura 8: a rede entra em **oscilação** (um “ciclo-limite”), balançando o nariz do avião sem parar, enquanto o PID rastreia limpo.

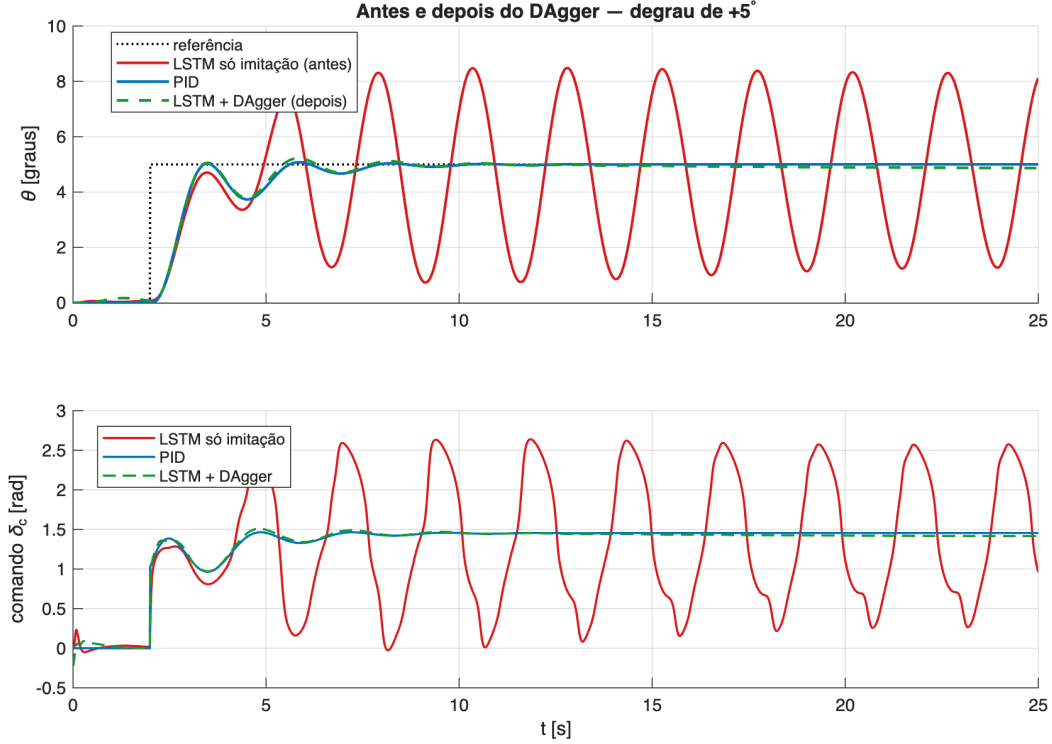


Figura 8: O efeito do DAgger, num degrau de $+5^\circ$. **Vermelho:** a rede de imitação pura oscila sem parar em malha fechada. **Azul:** o PID. **Verde tracejado:** a *mesma* rede após o DAgger — a oscilação desaparece e ela passa a acompanhar o PID.

A causa tem nome técnico: *desvio de distribuição* (em inglês, *distribution shift*). A imitação simples treina a rede *apenas* nas situações que o professor (o PID) percorre; mas, no comando, a rede gera situações *próprias* — justamente as que os erros dela criam — e nessas ela nunca foi treinada. Como nunca aprendeu a corrigi-las, os erros se acumulam até virar a oscilação.

O **DAgger** (do inglês *Dataset Aggregation*, “agregação de dados”; Ross *et al.* [1]) é um método *iterativo* que ataca exatamente isso. A cada rodada:

1. a rede *atual* comanda o avião, e nós registramos as situações (os erros $e(t)$) que **ela mesma** visita — inclusive as oscilatórias;
2. para cada uma dessas situações, perguntamos ao professor “o que você faria aqui?” e guardamos a resposta certa (o comando do PID);
3. **agregamos** esses novos exemplos ao banco — sem descartar os antigos, daí o nome “agregação” — e **treinamos de novo**.

A cada repetição, o banco passa a conter cada vez mais as situações que a rede *de fato* encontra, e ela aprende a se recuperar dos próprios erros. Um detalhe que torna o método viável aqui: o professor precisa saber responder em *qualquer* situação — e o PID, por ser uma fórmula ($\delta_c = K_p e + K_i \int e$), pode ser consultado em qualquer erro, sem custo. Fizemos **duas rodadas**; ao fim, a oscilação havia desaparecido (a curva verde da Fig. 8).

Para tornar o laço concreto, vale segui-lo no próprio caso do degrau de $+5^\circ$ (Fig. 8).

Passo 1 — o que se grava. Rodamos a malha fechada com a rede *atual* (a que oscila) no comando. A cada passo de tempo (100 Hz, ≈ 2500 pontos no episódio) gravamos **o erro que a malha visitou**,

$$e(t) = \theta_{\text{ref}}(t) - \theta(t).$$

Como a rede oscila, esse $e(t)$ também oscila — é o “espelho” da curva vermelha: quando a rede deixa θ subir a $8,5^\circ$, o erro vai a $5 - 8,5 = -3,5^\circ$; quando θ cai a $1,5^\circ$, o erro vai a $+3,5^\circ$. Guardamos o erro (a “situação”), que será a **entrada** do retreino; *não* guardamos o comando que a rede deu — esse é o comportamento ruim, que não queremos copiar. Só precisamos saber em *quais situações* ela se meteu.

Passo 2 — “o que você faria aqui?”. O professor é o PID, que é apenas a fórmula $\delta_c = K_p e + K_i \int e$. Então, para rotular cada situação, não re-simulamos nada: **aplicamos a fórmula do PID ao erro gravado**, instante a instante — com a mesma integração discreta do integrador do PID:

$$I[k] = I[k - 1] + T_s e[k - 1], \quad \delta_c^{\text{PID}}[k] = K_p e[k] + K_i I[k].$$

Ou seja, para *cada* instante da oscilação obtemos o comando correto. No exemplo: naquele momento em que θ havia passado para $8,5^\circ$ (erro $-3,5^\circ$), o rótulo do PID manda o comando *para baixo*, para derrubar o nariz — exatamente a ação corretiva que a rede oscilante não soube fazer. O novo exemplo de treino é o par de sequências

$$\underbrace{e(t)}_{\text{entrada (erro visitado)}} \longrightarrow \underbrace{\delta_c^{\text{PID}}(t)}_{\text{alvo (comando certo)}}.$$

Passo 3 — como entra no retreino. Esse par é **somado** ao banco original (os 240 episódios do PID) — agregação, não substituição — e a rede retreina no conjunto inteiro, com a mesma formulação sequência-a-sequência. A diferença é decisiva: *antes*, o banco só tinha trajetórias suaves do PID (que nunca oscila), e a rede nunca via “ θ passou do ponto e está oscilando $\rightarrow$ comando corretivo”; *depois*, o banco contém justamente essas situações oscilatórias rotuladas com a resposta amortecedora do PID. Ao reencontrar esses estados em operação, a rede agora sabe o que fazer — e a oscilação some.

Vale destacar que essa combinação — uma **rede LSTM treinada por imitação e refinada com DAgger** para substituir um controlador clássico — não é exclusiva deste trabalho. Espin *et al.* [2], por exemplo, usaram exatamente essa estratégia (rede recorrente LSTM + *behavior cloning* + DAgger) para emular um controlador preditivo (MPC) no carregamento ótimo de baterias, com a mesma motivação que a nossa: a imitação pura sofre de desvio de distribuição, e o DAgger o corrige realimentando os estados que a própria rede visita. O uso de redes recorrentes em aprendizado por imitação também é estudado de forma mais geral em [4].

5 Os dados de treinamento

A rede aprende por imitação, então tudo depende dos exemplos que mostramos a ela. Esta seção descreve *quais* dados foram usados e *como* foram gerados.

5.1 A ideia: o PID gera os dados

Não programamos a rede com regras. Em vez disso, deixamos o controlador PID comandar a planta em simulação e gravamos, a cada instante, um par

$$\underbrace{e(t) = \theta_{\text{ref}} - \theta}_{\text{entrada}} \longrightarrow \underbrace{\delta_c(t) \text{ (comando do PID)}}_{\text{alvo}}.$$

Esses pares “erro $\rightarrow$ comando” são os exemplos de treino: a rede deve, a partir do histórico do erro, produzir o mesmo comando que o PID produziu. A Figura 9 mostra um episódio: o PID controla a planta (acima), e dele extraímos a entrada $e(t)$ (meio) e o alvo $\delta_c(t)$ (abaixo), amostrados a 100 Hz.

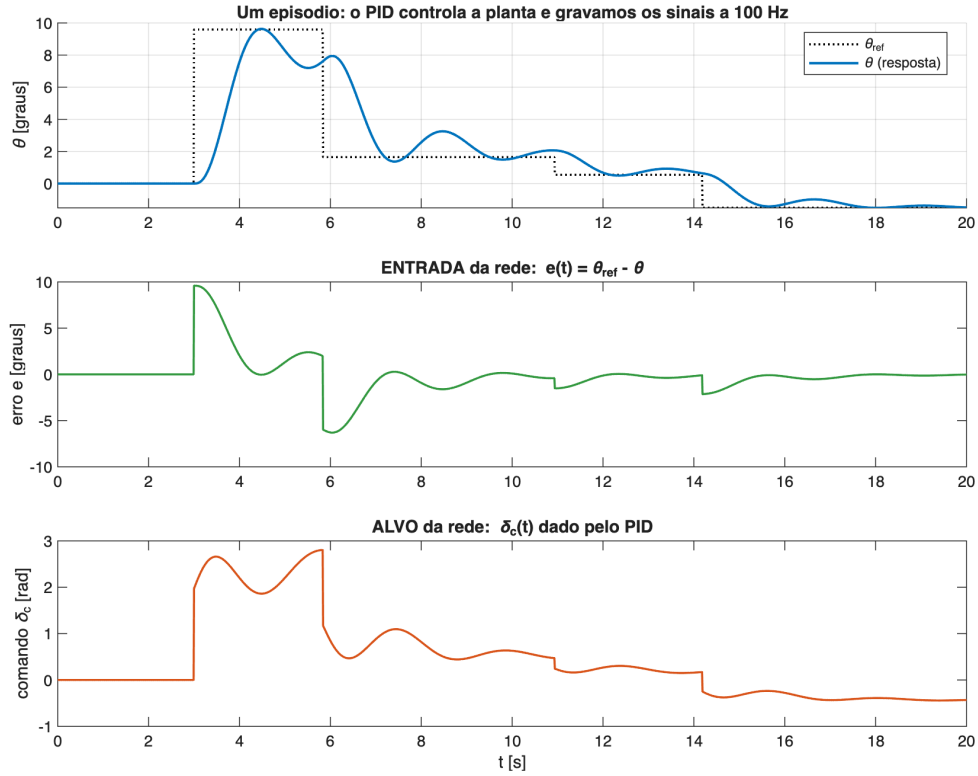


Figura 9: De cada simulação extraímos os sinais que treinam a rede. **Acima:** o PID segue a referência θ_{ref} . **Meio:** a *entrada* da rede, o erro $e = \theta_{ref} - \theta$. **Abaixo:** o *alvo*, o comando δ_c que o PID enviou. A rede aprende o mapa $e(t) \rightarrow \delta_c(t)$.

5.2 Variedade: cobrindo muitas situações

Para generalizar, a rede precisa ver muitas situações diferentes — não só um degrau. Por isso rodamos a malha fechada com **quatro famílias de referência** $\theta_{ref}(t)$, sorteadas aleatoriamente (Fig. 10):

- **degraus:** 2 a 4 saltos, em instantes e amplitudes aleatórios;
- **doublet:** um pulso $+A$ seguido de $-A$;
- **rampa:** subida gradual, patamar e descida;
- **multissenso:** soma de 3 a 5 senoides (0,05–0,5 Hz).

Todas as referências ficam dentro de $\pm 10^\circ$, e os episódios começam com a planta nivelada. Cada episódio dura 20 s a 100 Hz; geramos **240 episódios** (60 de cada tipo), totalizando cerca de **480 mil** pares de treino. Usou-se um *único* PID: com vários controladores discordantes, a regressão convergiria para uma média sem significado — a variedade necessária vem das *manobras*, não do professor.

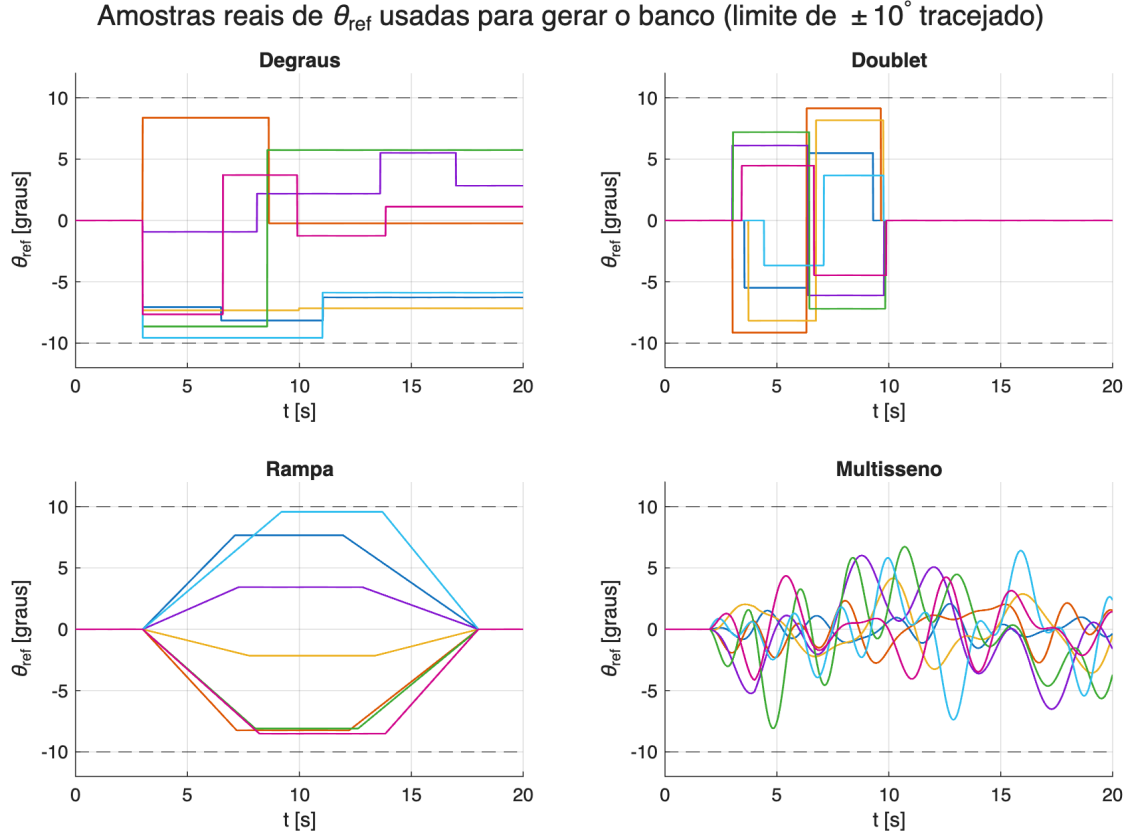


Figura 10: Amostras reais das quatro famílias de referência θ_{ref} usadas para gerar o banco (sete sorteios de cada, sobrepostos). Todas respeitam o limite de $\pm 10^\circ$ (linhas tracejadas) — a faixa de operação que a rede efetivamente aprende.

5.3 O reforço do DAgger

A Seção 4.2 explicou *por que* o DAgger é necessário (a imitação pura oscilava em malha fechada, Fig. 8) e *como* ele funciona. Aqui registramos apenas *quanto* ele acrescentou ao banco: a própria rede comandou a planta e, dos erros $e(t)$ que ela visitou (inclusive os estados oscilatórios que o PID nunca cria), geramos **136 episódios** extras, cada um rotulado com o comando exato do PID, $\delta_c = K_p e + K_i \int e$. Somados aos 240 originais, o banco final tem **376 episódios**.

6 Resultado: a rede empata com o controlador clássico

Depois desse treino, colocamos a rede neural para comandar o avião e comparamos com o controlador clássico nas mesmas situações. As Figuras 11 e 12 mostram um exemplo: pedimos uma inclinação de $+7^\circ$ e depois -4° . As duas curvas (clássico e rede neural) ficam praticamente **em cima uma da outra**.

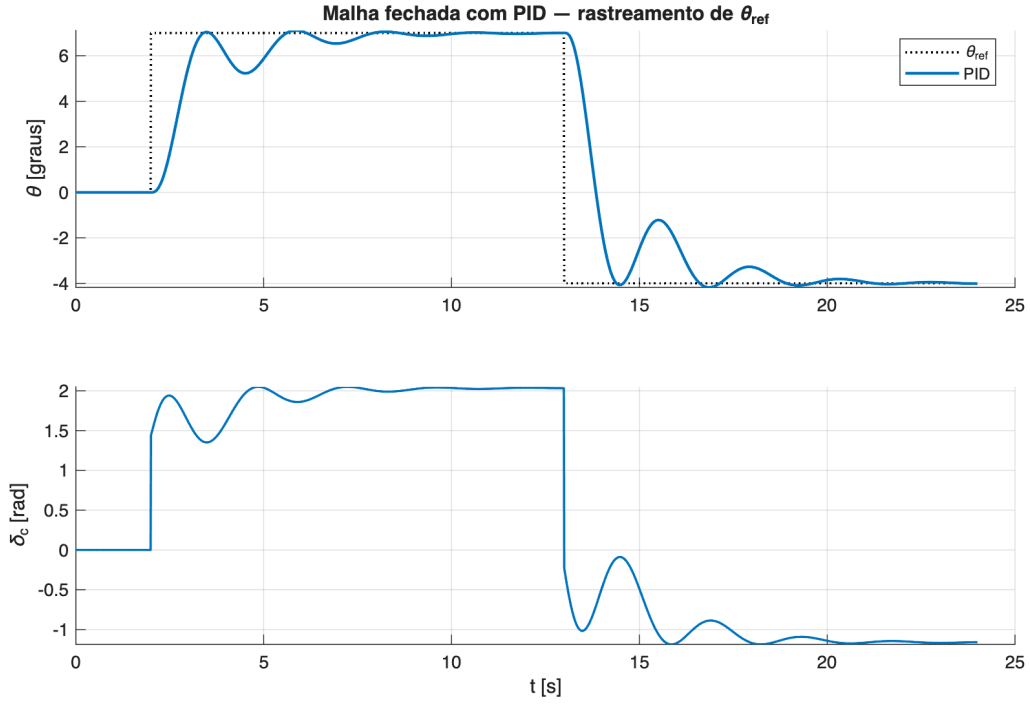


Figura 11: Avião comandado pelo **controlador clássico (PID)**. Acima: a inclinação (linha cheia) seguindo o alvo (linha pontilhada). Abaixo: o comando enviado ao profundor.

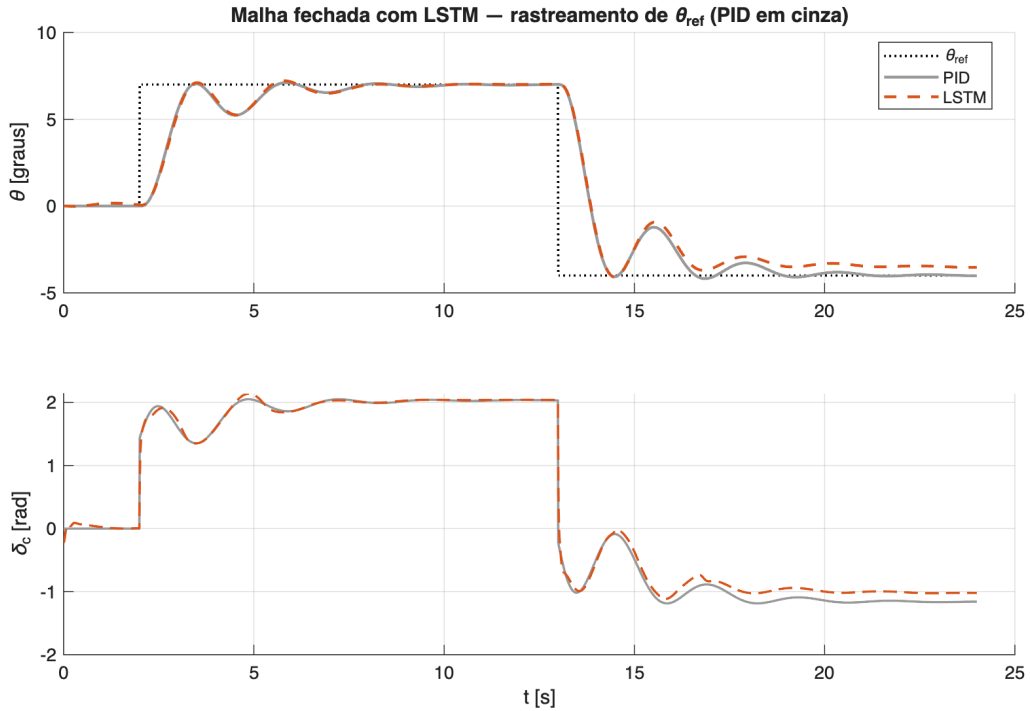


Figura 12: O mesmo avião comandado pela **rede neural** (em laranja), com o controlador clássico em cinza para comparação. As curvas quase se sobrepõem: a rede faz praticamente a mesma coisa que o controlador clássico.

A métrica de desempenho é o **RMSE de rastreamento** (raiz do erro quadrático médio entre θ e θ_{ref}). A Tabela 1 compara PID e LSTM em sete cenários, incluindo perfis não vistos no treino. A LSTM empata com o PID (RMSE mediano $0,91^\circ$ contra $0,92^\circ$), com desvio RMS entre as duas respostas de apenas $0,13^\circ$ — praticamente sobrepostas.

Tabela 1: Comparação PID $\times$ LSTM em sete cenários. RMSE de rastreamento (entre θ e θ_{ref}) e desvio RMS entre as duas respostas, em graus. († perfis não vistos no treino.)

Cenário	RMSE PID ($^\circ$)	RMSE LSTM ($^\circ$)	Desvio RMS LSTM–PID ($^\circ$)
degrau $+3^\circ$	0,47	0,49	0,13
degrau $+5^\circ$	0,79	0,79	0,08
degrau $+7^\circ$	1,10	1,11	0,05
degrau -5°	0,79	0,80	0,18
dois degraus	0,92	0,92	0,19
rampa †	0,93	0,91	0,15
multisseno †	1,93	1,89	0,11
mediana	0,92	0,91	0,13

A Figura 13 mostra a situação “subir 5° ”. Vale destacar: foi *exatamente* este caso que, antes do treino extra (Dagger), ficava oscilando sem parar. Depois do DAgger, a rede acompanha o controlador clássico perfeitamente.

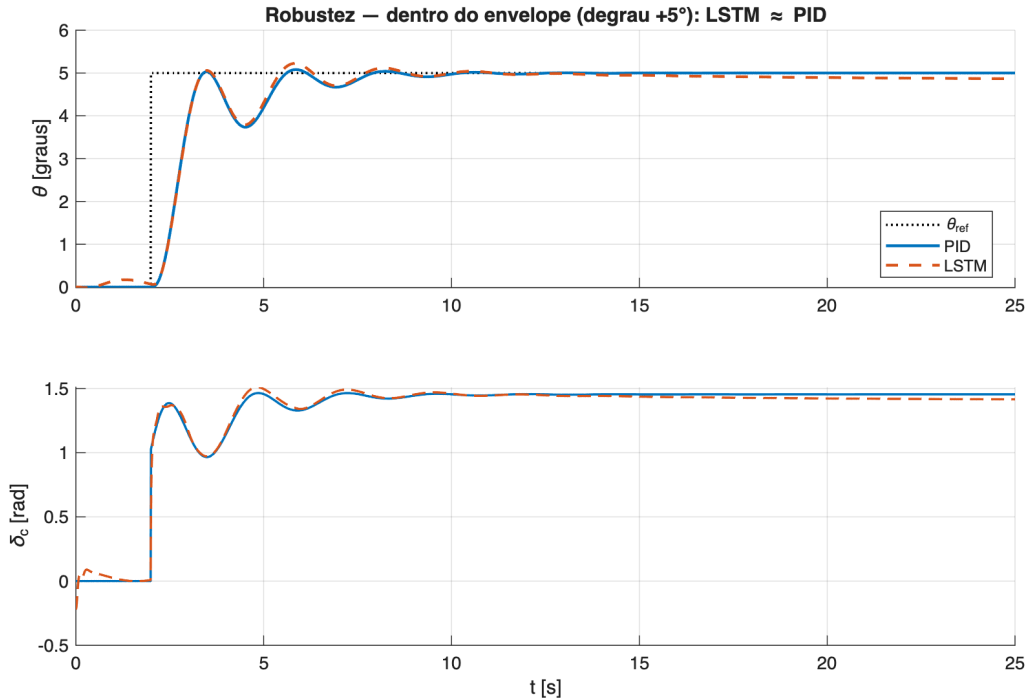


Figura 13: Subir 5° : rede neural (laranja) e controlador clássico (azul) são praticamente indistinguíveis. Antes do treino extra (Dagger), neste mesmo caso o avião ficava balançando $\pm 3,5^\circ$ sem parar.

7 Onde a rede falha: testando fora do que ela aprendeu

A rede foi treinada em situações “de rotina”: o avião começando nivelado e pedidos de inclinação de até 10° . O que acontece quando saímos disso? Testamos, e a resposta confirma a lição central: **a rede só é boa onde já viu situações parecidas**. Importante: em *todos* os testes, mesmo falhando, o avião nunca descontrolou de vez — a rede sempre se manteve estável.

(1) Começar fora da posição. Se o avião já *começa* inclinado (por exemplo, $+8^\circ$) e pedimos que volte a zero, o controlador clássico faz isso de forma suave. A rede também consegue voltar ao alvo — mas com um começo **muito agitado**, balançando bastante antes de se acalmar

(Figura 14). O motivo: durante o treino, o avião *sempre* começava nivelado; começar já inclinado é uma situação nova, e a rede leva alguns segundos para “se achar”.

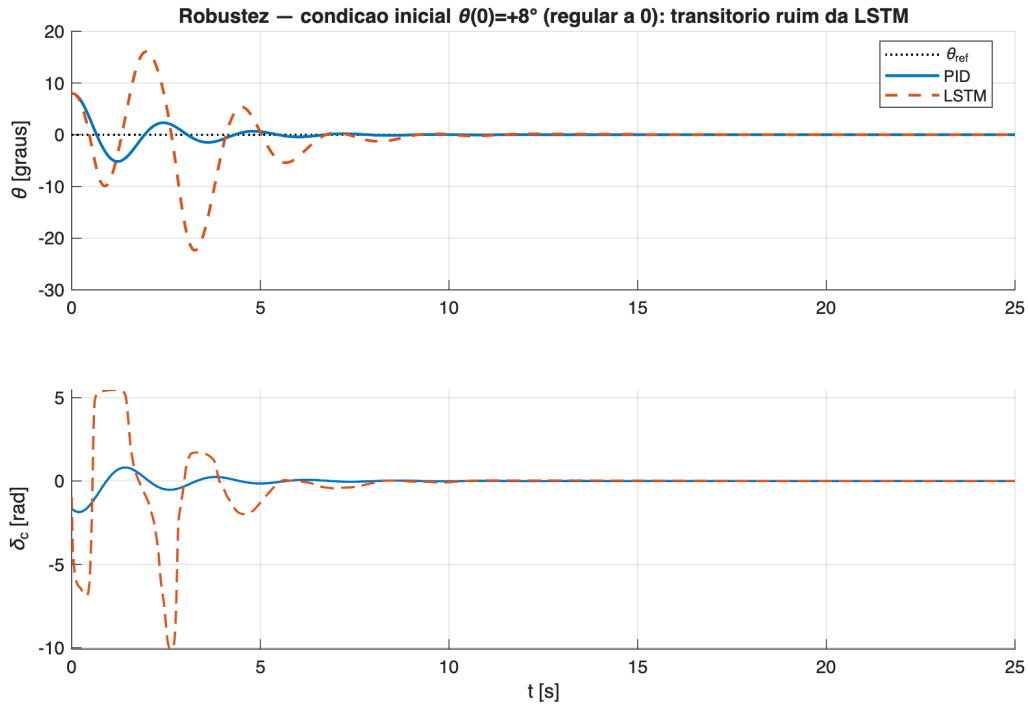


Figura 14: Avião começando inclinado em $+8^\circ$ e tendo que voltar a zero. O controlador clássico (azul) volta suave; a rede neural (laranja) chega lá, mas com um início bem agitado, porque nunca treinou começando fora da posição.

(2) **Pedir inclinações maiores do que as do treino.** A rede treinou com pedidos de até 10° . Quando pedimos 13° , ela **volta a oscilar** — o avião fica balançando em torno do alvo (Figura 15). O controlador clássico, que não “decora” e sim segue uma fórmula, acomoda normalmente. Esse é o limite mais claro da rede: fora da faixa que viu, ela falha.

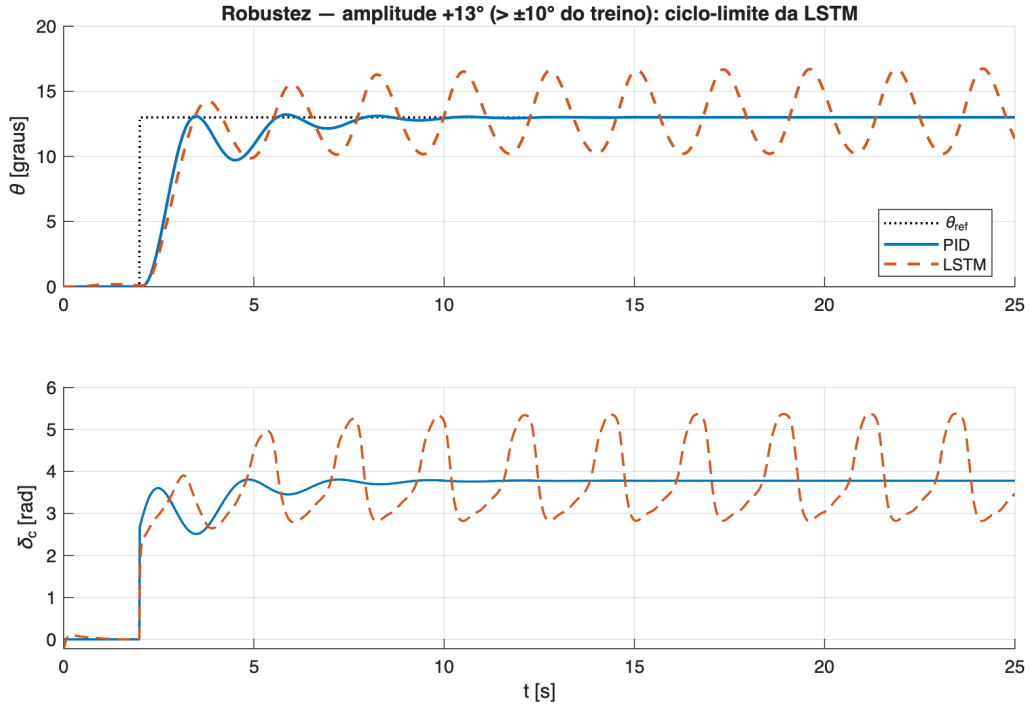


Figura 15: Pedido de 13° (acima dos 10° que a rede viu no treino): o controlador clássico (azul) acomoda limpo; a rede neural (laranja) fica oscilando sem parar.

(3) Pedir movimentos mais rápidos. Quando a inclinação desejada muda muito rápido, *os dois* controladores ficam para trás — mas igualmente (Figura 16). Aqui não é culpa da rede: é um limite físico do próprio avião, que não consegue acompanhar mudanças tão rápidas. A rede continua imitando bem o clássico.

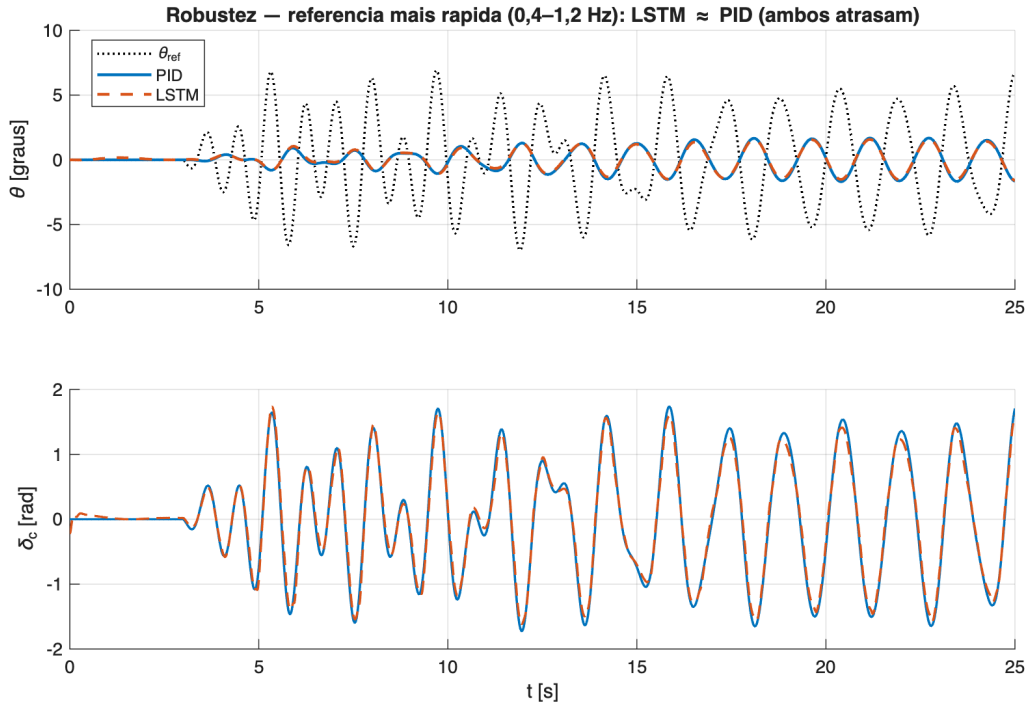


Figura 16: Pedido que muda muito rápido: rede neural (laranja) e controlador clássico (azul) reagem quase igual — ambos “atrasam” em relação ao alvo. É limite do avião, não da rede.

A Tabela 2 resume. Note que em todos os casos o avião permaneceu estável (não descontro-

lou).

Tabela 2: Testes fora do envelope de treino (RMSE de rastreamento, em graus). Em todos os casos a malha permaneceu estável.

Cenário	RMSE PID (°)	RMSE LSTM (°)	Estável?
$\theta(0) = +8^\circ, \theta_{\text{ref}} = 0$	1,30	4,90	sim
$\theta(0) = +10^\circ, \theta_{\text{ref}} = -5^\circ$	1,96	5,37	sim
degrau $+13^\circ$ (extrapolação)	2,05	2,92	sim
degrau -15° (extrapolação)	2,36	2,56	sim
referência rápida (0,4–1,2 Hz)	4,08	4,03	sim

8 O que pode ser melhorado

Os testes da seção anterior apontam, com clareza, onde o método pode crescer. As melhorias abaixo estão ordenadas da mais direta (e que resolveria a maior parte das falhas observadas) à mais ambiciosa.

1. **Ampliar o envelope de treino.** Quase todas as falhas vêm de a rede ser testada *fora* do que viu. A correção é direta: gerar os dados também com o avião **começando inclinado** ($\theta(0) \neq 0$), com **amplitudes maiores** que $\pm 10^\circ$ e com **referências mais rápidas**; em seguida, repetir o DAGger. A planta de simulação já permite começar fora da posição, então esse passo é barato e deve eliminar tanto a oscilação em ângulos grandes (Fig. 15) quanto boa parte do transitório agitado ao iniciar deslocado (Fig. 14).
2. **Resolver o “arranque a frio” da memória.** O pior caso — começar inclinado — piora porque o estado interno h da LSTM (Eq. (4)) sempre *parte do zero*, como se a rede “acordasse” sem saber em que situação o avião está. Além de treinar com $\theta(0) \neq 0$, poder-se-ia **informar a inclinação inicial à rede** (como uma entrada extra) ou dar a ela um breve intervalo de “aquecimento” antes de assumir o comando.
3. **Adicionar uma camada de segurança (supervisor).** Como a rede não tem garantia fora do envelope, um esquema prático é **monitorar** se a situação atual está dentro da faixa conhecida e, em caso negativo, **devolver o comando ao PID** (ou limitar/saturar a ação da rede). Assim combina-se o melhor dos dois mundos: a rede onde ela é boa, e a garantia clássica como rede de proteção.
4. **Testar robustez a variações da planta.** Toda a avaliação usou um *único* modelo de avião. Na prática, massa, velocidade e centro de gravidade mudam. Convém verificar como a rede se comporta quando os parâmetros da planta variam — e, se necessário, treiná-la com uma *família* de plantas (treino com variação), algo que o controlador clássico tolera por projeto.
5. **Incluir ruído de sensor.** Os dados de treino são “limpos”. Medidas reais de θ têm ruído; treinar com ruído (e talvez penalizar comandos muito “nervosos”) tornaria a rede mais realista e suave.
6. **Buscar garantias formais de estabilidade.** A diferença de fundo para o PID é que este tem **prova matemática** de estabilidade, e a rede não. Existem técnicas recentes para *certificar* controladores neurais (análise por funções de Lyapunov, restrições integrais quadráticas / IQC, verificação formal de redes). Aplicá-las aqui transformaria a “garantia na prática” numa garantia de fato — um próximo passo natural e de maior fôlego.

7. **Enxugar a rede.** Para embarcar em hardware real, valeria estudar redes menores ou alternativas (por exemplo *GRU*, uma variante mais simples da LSTM) e técnicas de compressão, medindo o *trade-off* entre tamanho e qualidade do controle.

9 Conclusão

Principais conclusões deste estudo:

- **Dá para trocar o controlador clássico por uma rede neural** e obter o mesmo desempenho — desde que a rede seja treinada do jeito certo.
- **Só copiar o professor não é suficiente.** A rede parecia ótima “no papel”, mas balançava o avião quando assumia o comando. O treino extra (Dagger), em que ela aprende com os próprios erros, resolveu isso.
- **A rede só é confiável no que já viu.** Dentro da faixa de treino, ela empata com o clássico. Fora dela — começando muito inclinada ou com pedidos maiores — ela piora, embora sem descontrolar. O controlador clássico, por seguir uma fórmula, não tem esse limite.

A diferença essencial é essa: o controlador clássico tem **garantia matemática** de funcionar em qualquer situação; a rede neural tem apenas **garantia na prática**, e só onde foi treinada. Para ampliar essa faixa, bastaria treinar a rede também em situações mais variadas (começando inclinada, com pedidos maiores) e repetir o Dagger — o que mostraria, na prática, que o limite observado é de *falta de exemplos*, não do método.

Em resumo: uma rede neural pode, sim, substituir um controlador clássico de avião com a mesma qualidade — contanto que a gente treine direito e respeite o seu limite: ela é tão boa quanto os exemplos que viu.

Referências

- [1] S. Ross, G. Gordon, J. A. Bagnell. *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning* (Dagger). *Proceedings of AISTATS*, 2011.
- [2] J. Espin, D. Zhang, D. Toti, A. Pozzi. *Deep-MPC: A DAGGER-Driven Imitation Learning Strategy for Optimal Constrained Battery Charging*. arXiv:2406.15985, 2024.
- [3] S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory*. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] K. Nguyen. *Imitation Learning with Recurrent Neural Networks*. arXiv:1607.05241, 2016.